SPFx Launcher Plan: Addendum

Pluggable Architecture, Framework Integration & Platform Support

Document Status: Addendum to SPFx.Launcher.Plan.Preview.md
Date: October 1, 2026
Based on: SPA_PWA_SPFX_Pluggable_Framework_ChatGPT_2026-09-30_Wednesday.md

---

Table of Contents

1. Executive Summary
2. Pluggable Architecture Deep Dive
    - 2.1 Event Bus Implementation
    - 2.2 Persisted & Shared Data
    - 2.3 Modern Web Part Compatibility
    - 2.4 Shared Logic & Code Reuse
3. Security Context & Authentication
    - 3.1 SPFx Security Model
    - 3.2 Isolation & Sandboxing
4. TypeScript Transpilation for Plugins
    - 4.1 Compilation Strategy
    - 4.2 No-Build Plugin Architecture
    - 4.3 JavaScript + JSDoc Alternative
5. Fluent UI: Design System & Component Library
    - 5.1 Fluent UI React vs. Fluent Web Components
    - 5.2 Styling & Theming
6. Starting Small: Minimal Pluggable Applet
    - 6.1 Proof of Concept
    - 6.2 Incremental Growth Strategy
7. SPAs vs. True Distributed Applications
    - 7.1 Single Page App (SPA)
    - 7.2 Progressive Web App (PWA)
    - 7.3 Micro Frontend Architecture
8. Outlook Office Add-Ins: Architecture & Deployment
    - 8.1 Outlook Add-In Model
    - 8.2 Task Pane Add-Ins
    - 8.3 Deployment Scenarios
9. Markdown & Mermaid Renderer for Legacy Outlook
    - 9.1 Architecture
    - 9.2 Technical Implementation
    - 9.3 Key Considerations
10. Web Components in Modern Browsers
    - 10.1 Web Components Fundamentals
    - 10.2 Web Components vs. React Components
    - 10.3 SPFx Web Components Support
    - 10.4 Lit Framework for Web Components
11. SPFx Platform Support Matrix
    - 11.1 Comprehensive Support Table
    - 11.2 Platform-Specific Notes
12. Architectural Decision Matrix
13. Recommended Implementation Path
14. Technical Glossary (Addendum)

Executive Summary

↑ Back to Top

This addendum builds on **SPFx.Launcher.Plan.Preview.md** to provide:

1. **Pluggable Architecture Blueprint** — Event buses, shared data stores, and inter-plugin communication patterns
2. **Security & Context Management** — How SPFx contexts are isolated and shared
3. **TypeScript Build Strategy** — Centralized compilation vs. no-build JavaScript plugins
4. **UI Framework Analysis** — Fluent UI React, Web Components, and Lit
5. **Minimal Starting Point** — Single pluggable applet approach before scaling
6. **Platform Coverage** — Complete matrix of SPFx support across Microsoft 365 apps
7. **Office Add-In Strategy** — Including Outlook desktop client for Markdown/Mermaid rendering

Key Insight: Rather than maintaining separate SPFx projects, deploy **one container** that loads plugins dynamically. This reduces deployment friction and enables code reuse across SharePoint, Teams, Outlook, and Power Apps.

↑ Back to Top

Pluggable Architecture Deep Dive

↑ Back to Top

2.1 Event Bus Implementation

↑ Back to Top

The event bus is the **nervous system** of a pluggable architecture. It allows plugins to communicate without direct coupling.

Core Event Bus Pattern:

```
// services/eventBus.ts
import { EventEmitter } from 'events';

export interface FrameworkEvent {
  type: string;
  sourcePluginId: string;
  payload: any;
  timestamp: number;
  priority?: 'low' | 'normal' | 'high';
}

export class FrameworkEventBus extends EventEmitter {
  private eventHistory: FrameworkEvent[] = [];
  private maxHistorySize = 100;

  publish(event: Omit<FrameworkEvent, 'timestamp'>): void {
    const fullEvent: FrameworkEvent = {
      ...event,
      timestamp: Date.now()
    };

    // Publish immediately
    this.emit(event.type, fullEvent);

    // Keep history for debugging/replay
    this.eventHistory.push(fullEvent);
    if (this.eventHistory.length > this.maxHistorySize) {
      this.eventHistory.shift();
    }

    // Log high-priority events
    if (fullEvent.priority === 'high') {
      console.log(`[EVENT] ${event.type}:`, fullEvent.payload);
    }
  }

  subscribe(
    eventType: string,
    handler: (event: FrameworkEvent) => void,
    options?: { once?: boolean; priority?: number }
  ): () => void {
    if (options?.once) {
      this.once(eventType, handler);
    } else {
      this.on(eventType, handler);
    }

    // Return unsubscribe function
    return () => this.off(eventType, handler);
  }

  getHistory(eventType?: string): FrameworkEvent[] {
    if (!eventType) return [...this.eventHistory];
    return this.eventHistory.filter(e => e.type === eventType);
  }
}
```

Plugin Usage Example:

```
// plugins/site-editor/index.ts
export async function initialize(context: ISharedContext) {
  // Subscribe to site selection
  context.eventBus.subscribe('site:selected', async (event) => {
    const { siteUrl } = event.payload;
    const siteData = await fetchSiteDetails(siteUrl);

    // Publish updated site info
    context.eventBus.publish({
      type: 'site:loaded',
      sourcePluginId: 'site-editor',
      payload: { site: siteData },
      priority: 'high'
    });
  });

  // Subscribe to save requests
  const unsubscribeSave = context.eventBus.subscribe(
    'save:requested',
    async (event) => {
      await saveSiteChanges(event.payload);
    }
  );

  return { unsubscribeSave };
}
```

Standard Event Types:

Event	Source	Payload	Purpose
site:selected	Site Picker plugin	{ siteUrl, siteId }	Notify when user selects a site
site:loaded	Site Details plugin	{ site: ISiteInfo }	Announce site data fetched
selection:changed	Any plugin	{ itemId, itemType }	Generic selection notification
data:modified	Any plugin	{ entityType, action, data }	Data mutation event
save:requested	UI	{ changes, context }	Trigger save operation
sync:required	Any plugin	{ reason, data }	Force sync across instances
error:occurred	Any plugin	{ error, context, recoverable }	Error notification
workflow:updated	Workflow plugin	{ flowId, status, result }	Workflow state change

↑ Back to Top

2.2 Persisted & Shared Data

↑ Back to Top

Multi-Level Data Architecture:

```
┌─────────────────────────────────────────┐
│  User Browser Session                   │
├─────────────────────────────────────────┤
│  sessionStorage (5 MB) - Temp cache     │
│  localStorage (5 MB) - Preferences      │
│  IndexedDB (hundreds of MB) - Structured data
│  Cache API (varies) - HTTP responses    │
├─────────────────────────────────────────┤
│  BroadcastChannel - Multi-tab sync      │
├─────────────────────────────────────────┤
│  Shared Data Service (Memory)           │
│  ├─ Current selections                  │
│  ├─ User identity                       │
│  ├─ Session tokens                      │
│  └─ Recent searches                     │
├─────────────────────────────────────────┤
│  Cloud (Authoritative)                  │
│  ├─ SharePoint Lists                    │
│  ├─ Dataverse                           │
│  ├─ SQL Server                          │
```

```
│  └─ REST APIs                          │
└──────────────────────────────────────┘
```

Shared Data Service Implementation:

```typescript
// services/sharedDataService.ts
export class SharedDataService {
  private store = new Map<string, any>();
  private subscriptions = new Map<string, Set<(data: any) => void>>();

  set(key: string, value: any): void {
    const oldValue = this.store.get(key);
    this.store.set(key, value);

    // Notify subscribers
    if (this.subscriptions.has(key)) {
      this.subscriptions.get(key)!.forEach(callback => {
        callback(value);
      });
    }

    // Persist to localStorage for simple data
    if (typeof value === 'string' || typeof value === 'number') {
      try {
        localStorage.setItem(`shared_${key}`, JSON.stringify(value));
      } catch (e) {
        console.warn('localStorage quota exceeded', e);
      }
    }
  }

  get<T = any>(key: string): T | undefined {
    return this.store.get(key) as T | undefined;
  }

  subscribe(key: string, callback: (data: any) => void): () => void {
    if (!this.subscriptions.has(key)) {
      this.subscriptions.set(key, new Set());
    }
    this.subscriptions.get(key)!.add(callback);

    // Return unsubscribe function
    return () => {
      this.subscriptions.get(key)?.delete(callback);
    };
  }

  // For distributed data across browser tabs
  broadcastChange(key: string, value: any): void {
    const channel = new BroadcastChannel('shared-data');
    channel.postMessage({ key, value, source: 'shared-data-service' });
    channel.close();
  }

  // Cache API for large datasets
  async setCachedData(cacheKey: string, data: any): Promise<void> {
    if ('caches' in window) {
      const cache = await caches.open('app-data');
      const response = new Response(JSON.stringify(data), {
        headers: { 'Content-Type': 'application/json' }
      });
      await cache.put(cacheKey, response);
    }
  }

  async getCachedData(cacheKey: string): Promise<any | null> {
    if ('caches' in window) {
      const cache = await caches.open('app-data');
      const response = await cache.match(cacheKey);
      if (response) {
        return response.json();
      }
    }
    return null;
  }
}
```

Plugin Data Sharing Example:

```
// In multiple plugins
const unsubscribe = context.dataService.subscribe('currentSite', (site) => {
  console.log('Site changed to:', site.title);
  updateUI(site);
});

// Another plugin updates the shared data
context.dataService.set('currentSite', {
  id: '123',
  title: 'Sales Hub',
  url: 'https://...'
});
```

↑ Back to Top

2.3 Modern Web Part Compatibility

↑ Back to Top

A modern web part in SharePoint Online is a React component wrapped by SPFx. Your launcher should support both direct mounting and compatibility layers.

Web Part as Plugin:

```
// Modern Web Part Definition
export interface IModernWebPart {
  id: string;
  title: string;
  description?: string;
  webPartData?: any; // Web part properties
  render(container: HTMLElement, context: ISharedContext): Promise<void>;
  dispose(): Promise<void>;
}

// Compatibility layer for reusing existing web parts as plugins
export class WebPartPlugin implements IModernWebPart {
  private webPartInstance: any;

  constructor(
    private webPartClass: any,
    private properties: any
  ) {}

  async render(container: HTMLElement, context: ISharedContext): Promise<void> {
    // Create web part instance
    this.webPartInstance = new this.webPartClass();

    // Initialize with properties
    this.webPartInstance.onInit(this.properties, context);

    // Render to container
    await this.webPartInstance.render(container);
  }

  async dispose(): Promise<void> {
    if (this.webPartInstance?.dispose) {
      await this.webPartInstance.dispose();
    }
  }
}
```

SharePoint List Compatibility:

```
// Display a SharePoint list within a plugin
export async function renderListPlugin(
  listName: string,
  container: HTMLElement,
  context: ISharedContext
) {
  const items = await context.spClient.getListItems(listName);

  const listElement = document.createElement('div');
  listElement.className = 'list-container';
  listElement.innerHTML = `
```

```
    <table class="list-table">
      <thead>
        <tr>
          <th>Title</th>
          <th>Status</th>
          <th>Modified</th>
        </tr>
      </thead>
      <tbody>
        ${items.map(item => `
          <tr>
            <td>${item.Title}</td>
            <td>${item.Status || 'N/A'}</td>
            <td>${new Date(item.Modified).toLocaleDateString()}</td>
          </tr>
        `).join('')}
      </tbody>
    </table>
  `;

  container.appendChild(listElement);
}
```

↑ Back to Top

2.4 Shared Logic & Code Reuse

↑ Back to Top

Common Services Shared Across Plugins:

```
// services/commonServices.ts
export interface ICommonServices {
  // Authentication
  getCurrentUser(): Promise<IUser>;
  getAuthToken(scopes: string[]): Promise<string>;
  switchAccount(accountType: 'regular' | 'admin'): Promise<void>;

  // SharePoint
  getSite(siteUrl: string): Promise<ISiteInfo>;
  getListItems(
    siteUrl: string,
    listId: string,
    filter?: string
  ): Promise<IListItem[]>;
  updateListItem(
    siteUrl: string,
    listId: string,
    itemId: number,
    updates: any
  ): Promise<IListItem>;

  // Microsoft Graph
  searchUsers(query: string): Promise<IUser[]>;
  getDirectReports(userId: string): Promise<IUser[]>;
  getUserPhoto(userId: string): Promise<Blob>;

  // Dataverse
  queryDataverse(
    environment: string,
    query: string
  ): Promise<any[]>;

  // Utilities
  notify(message: string, type: 'info' | 'success' | 'warning' | 'error'): void;
  openDialog(title: string, content: string): Promise<boolean>;
  downloadFile(filename: string, data: Blob): void;
  log(message: string, level?: 'debug' | 'info' | 'warn' | 'error'): void;
}
```

Shared UI Components Library:

```
// components/shared/
export class SiteCard extends HTMLElement {
```

```
  connectedCallback() {
    const site = JSON.parse(this.dataset.site || '{}');
    this.render(site);
  }

  private render(site: any) {
    this.innerHTML = `
      <div class="site-card">
        <h3>${site.title}</h3>
        <p class="url">${site.url}</p>
        <p class="status">${site.status}</p>
      </div>
    `;
  }
}

export class PersonCard extends HTMLElement {
  connectedCallback() {
    const person = JSON.parse(this.dataset.person || '{}');
    this.render(person);
  }

  private render(person: any) {
    this.innerHTML = `
      <div class="person-card">
        <img src="${person.photoUrl}" alt="${person.displayName}">
        <h4>${person.displayName}</h4>
        <p>${person.jobTitle}</p>
      </div>
    `;
  }
}

customElements.define('site-card', SiteCard);
customElements.define('person-card', PersonCard);
```

↑ Back to Top

Security Context & Authentication

↑ Back to Top

3.1 SPFx Security Model

↑ Back to Top

SPFx provides a **secure context** through the page context:

```
// SPFx passes secure context to your app
export interface ISecurePageContext {
  user: {
    displayName: string;
    loginName: string; // UPN
    email: string;
  };
  site: {
    id: string;
    serverRelativeUrl: string;
  };
  web: {
    id: string;
    serverRelativeUrl: string;
  };
  site: {
    absoluteUrl: string;
  };
}
```

Key Security Features:

- **User Identity**: Authenticated user from Azure AD/Entra
- **Token Management**: MSAL tokens automatically refreshed
- **API Permissions**: Scopes granted via app registration
- **CSP Headers**: Content Security Policy for XSS protection

- **Same-Origin Policy**: Enforced by browser
- **CORS**: Managed by SharePoint/Microsoft Graph endpoints

Plugin Access Control:

```
// Plugins declare required permissions
export const ADDON_PERMISSIONS = {
  id: 'site-admin-plugin',
  requiredScopes: [
    'Sites.Read.All',
    'Sites.Manage.All'
  ],
  minimumRoles: ['SharePoint Administrator', 'Global Administrator'],
  dataAccess: ['SharePoint', 'Graph']
};

// Runtime permission check
export async function canAccessPlugin(
  userId: string,
  pluginId: string,
  context: ISharedContext
): Promise<boolean> {
  const roles = await context.graphClient.getUserMembershipIds(userId);
  const plugin = ADDON_PERMISSIONS;

  return plugin.minimumRoles.some(role =>
    roles.includes(role)
  );
}
```

↑ Back to Top

3.2 Isolation & Sandboxing

↑ Back to Top

Plugin Isolation Strategy:

```
// Each plugin runs with limited context
export interface IIsolatedPluginContext {
  // Limited data access - only what plugin declares
  sharePoint: {
    getCurrentSite(): Promise<ISiteInfo>;
    getListItems(listId: string): Promise<IListItem[]>;
  };

  // No direct DOM access beyond assigned container
  container: HTMLElement;

  // Mediated events - only published events
  events: IEventBus;

  // Sandboxed storage - plugin-specific namespace
  storage: {
    get(key: string): any;
    set(key: string, value: any): void;
  };
}
```

Container-Based Isolation:

```
// Shadow DOM prevents CSS/DOM leakage
export function createIsolatedPluginContainer(
  pluginId: string
): HTMLElement {
  const hostElement = document.createElement('div');
  hostElement.id = `plugin-${pluginId}`;

  // Create shadow root for style isolation
  const shadowRoot = hostElement.attachShadow({ mode: 'open' });

  // Add styles scoped to shadow DOM
  const style = document.createElement('style');
```

```
  style.textContent = `
    :host {
      display: block;
      /* Only affects this plugin's styles */
    }
  `;
  shadowRoot.appendChild(style);

  return hostElement;
}
```

↑ Back to Top

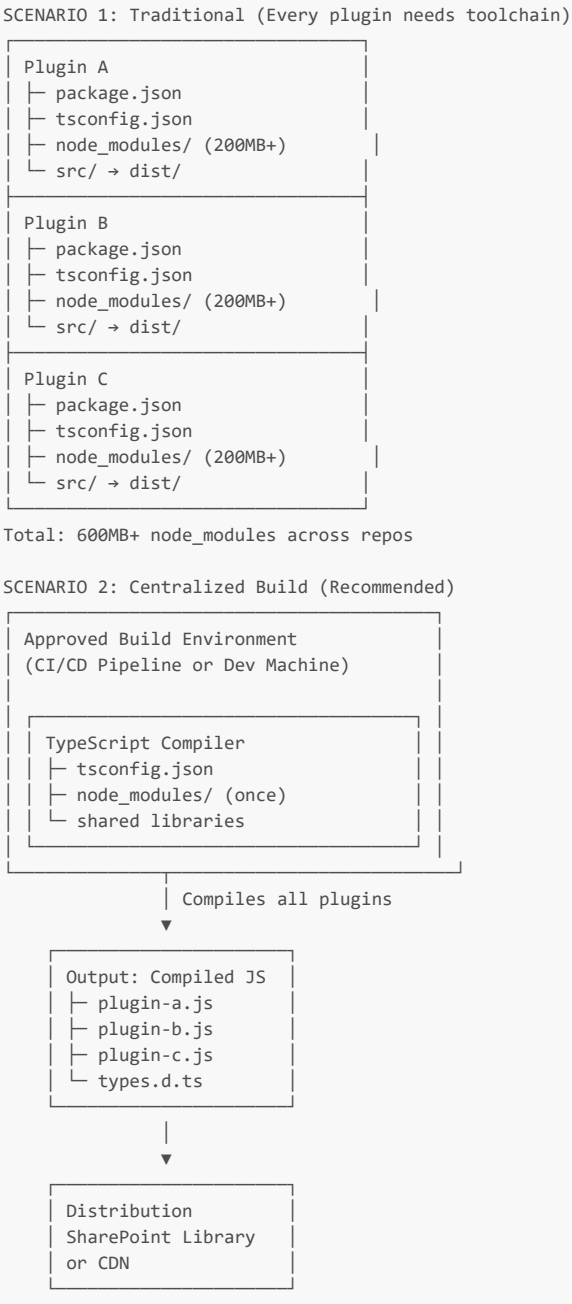
TypeScript Transpilation for Plugins

↑ Back to Top

4.1 Compilation Strategy

↑ Back to Top

Centralized Build vs. Distributed Plugins:



Esbuild Configuration for Plugins:

```
// build.js - Run once in approved environment
import * as esbuild from 'esbuild';
import fs from 'fs';
import path from 'path';

const pluginDir = './plugins';
const outDir = './dist';

const plugins = fs.readdirSync(pluginDir)
  .filter(f => fs.statSync(path.join(pluginDir, f)).isDirectory());

for (const plugin of plugins) {
  esbuild.buildSync({
    entryPoints: [path.join(pluginDir, plugin, 'index.ts')],
    bundle: true,
    outfile: path.join(outDir, `${plugin}.js`),
    format: 'es',
    target: 'es2020',
    external: ['react', 'react-dom'], // Shared by host
    minify: true,
    sourcemap: true,
    define: {
      'process.env.NODE_ENV': '"production"'
    }
  });

  console.log(`✓ Built ${plugin}`);
}
```

↑ Back to Top

4.2 No-Build Plugin Architecture

↑ Back to Top

Alternative: JavaScript-Only Plugins

If IT restricts TypeScript at the plugin level, use JavaScript with type hints:

```
// plugin.js - No build step needed
/**
 * @typedef {Object} SiteInfo
 * @property {string} id
 * @property {string} title
 * @property {string} url
 * @property {string} status
 */

/**
 * Get site information
 * @param {string} siteUrl - SharePoint site URL
 * @returns {Promise<SiteInfo>}
 */
export async function getSiteInfo(siteUrl) {
  const response = await fetch(`${siteUrl}/_api/web`);
  const data = await response.json();

  return {
    id: data.Id,
    title: data.Title,
    url: siteUrl,
    status: 'Active'
  };
}

export const manifest = {
  id: 'site-info-plugin',
  title: 'Site Information',
  version: '1.0'
};

/**
 * @param {Object} context - Shared context
 * @param {HTMLElement} context.container
```

```
 */
export async function initialize(context) {
  context.container.innerHTML = '<p>Initializing...</p>';
}
```

↑ Back to Top

4.3 JavaScript + JSDoc Alternative

↑ Back to Top

VS Code understands JSDoc well enough for good DX:

```
// @ts-check
// Enable TypeScript-like checking in VS Code without compiling

/**
 * @typedef {Object} IUserContext
 * @property {string} userId
 * @property {string} displayName
 * @property {string[]} roles
 */

/**
 * @param {IUserContext} user
 * @returns {Promise<boolean>}
 */
async function hasAdminRole(user) {
  return user.roles.includes('Administrator');
}
```

Benefits:

- No build step
- VS Code intellisense works
- Works in modern browsers (all ES modules)
- Can still add tsc type check to CI/CD pipeline
- Developers use `.js` files directly

↑ Back to Top

Fluent UI: Design System & Component Library

↑ Back to Top

5.1 Fluent UI React vs. Fluent Web Components

↑ Back to Top

Fluent UI React (v8/v9):

- Tightly integrated with React
- Rich component library (50+ components)
- Microsoft-designed, Microsoft-updated
- Default for SPFx projects
- Requires React as dependency

```
import { PrimaryButton, Panel, Stack } from '@fluentui/react';

export const SiteAdmin = () => (
  <Stack>
    <h1>Site Administration</h1>
    <PrimaryButton onClick={handleLock}>
      Lock Site
    </PrimaryButton>
  </Stack>
);
```

Fluent Web Components (Modern Web Components):

- Framework-agnostic

- Based on Web Components standard
- Can be used with React, Vue, Angular, plain JS
- Lighter weight
- Can be consumed via CDN

```
<!-- Use in plain HTML -->
<script type="module" src="https://cdn.jsdelivr.net/.../fluent-button.js"></script>

<fluent-button>
  Lock Site
</fluent-button>
```

Decision Matrix:

Aspect	Fluent UI React	Fluent Web Components
Framework	React-only	Framework-agnostic
Bundle size	~200 KB	~150 KB per component
Build required	Yes	No (CDN works)
TypeScript support	Excellent	Good (with JSDoc)
Plugin compatibility	Good if React	Excellent
Browser support	Modern browsers	Modern browsers
Microsoft support	Primary	Growing

Recommendation for Plugins: Use **Fluent Web Components** if you want plugins to be buildless and framework-agnostic.

↑ Back to Top

5.2 Styling & Theming

↑ Back to Top

Shared Theming Strategy:

```
// theme.ts - Define organizational theme once
export const organizationTheme = {
  palette: {
    themePrimary: '#0078d4', // Microsoft blue
    themeSecondary: '#107c10', // Microsoft green
    themeTertiary: '#50e6ff', // Cyan
    themeDark: '#004b50',

    neutralLightest: '#f3f2f1',
    neutralLight: '#edebe9',
    neutralLightAlt: '#e1dfdd',
    neutral: '#8a8886',
    neutralDark: '#605e5c',
    neutralDarker: '#3b3a39'
  },
  spacing: {
    s1: '4px',
    s2: '8px',
    s3: '12px',
    s4: '16px',
    s5: '24px'
  },
  typography: {
    fontFamily: '"Segoe UI", sans-serif',
    fontSize: {
      small: '12px',
      body: '14px',
      large: '16px',
      xLarge: '20px',
      xxLarge: '28px'
    }
  }
};

// Share with all plugins
context.theme = organizationTheme;
```

CSS-in-JS for Dynamic Styling:

```
import { css } from '@emotion/react';

export const pluginStyles = css`
  .plugin-container {
    background-color: ${organizationTheme.palette.neutralLightest};
    padding: ${organizationTheme.spacing.s4};
    font-family: ${organizationTheme.typography.fontFamily};
    font-size: ${organizationTheme.typography.fontSize.body};
    color: ${organizationTheme.palette.neutral};
  }

  .plugin-button {
    background-color: ${organizationTheme.palette.themePrimary};
    color: white;
    border: none;
    padding: ${organizationTheme.spacing.s3} ${organizationTheme.spacing.s4};
    border-radius: 4px;
    cursor: pointer;

    &:hover {
      background-color: ${organizationTheme.palette.themeSecondary};
    }
  }
`;
```

↑ Back to Top

Starting Small: Minimal Pluggable Applet

↑ Back to Top

6.1 Proof of Concept

↑ Back to Top

Minimal Host Web Part (SPFx):

```
import * as React from 'react';
import { WebPartContext } from '@microsoft/sp-webpart-base';

interface IPlugin {
  id: string;
  name: string;
  render(container: HTMLElement, context: any): Promise<void>;
}

export interface ILauncherState {
  plugins: IPlugin[];
  activePlugin: IPlugin | null;
}

export class LauncherWebPart extends React.Component<any, ILauncherState> {
  constructor(props: any) {
    super(props);
    this.state = {
      plugins: [],
      activePlugin: null
    };
  }

  async componentDidMount() {
    // Load plugins dynamically
    const plugins = await this.loadPlugins();
    this.setState({ plugins });
  }

  async loadPlugins(): Promise<IPlugin[]> {
    // Load from config
    const response = await fetch('./plugins.json');
    const config = await response.json();

    return Promise.all(
      config.plugins.map(async (p: any) => {
        const module = await import(p.module);
        return {
          id: p.id,
          name: p.name,
          render: module.render
        };
      })
    );
  }

  render() {
    return (
      <div>
        <div className="plugin-nav">
          {this.state.plugins.map(p => (
            <button key={p.id} onClick={() => this.setState({ activePlugin: p })}>
              {p.name}
            </button>
          ))}
        </div>
        <div id="plugin-container" className="plugin-workspace">
          {/* Active plugin renders here */}
        </div>
      </div>
    );
  }
}
```

Minimal Plugin (Plain JavaScript):

```
// plugins/hello-world/index.js
export const manifest = {
  id: 'hello-world',
  name: 'Hello World Plugin',
  version: '1.0'
};
```

```
export async function render(container, context) {
  container.innerHTML = `
    <div class="plugin">
      <h2>Hello World</h2>
      <p>Current user: ${context.user.displayName}</p>
      <p>Site: ${context.site.title}</p>
      <button onclick="alert('Plugin works!')">Test</button>
    </div>
  `;
}
```

↑ Back to Top

6.2 Incremental Growth Strategy

↑ Back to Top

Phase 1: Single Hello-World Plugin

- [ ] Create SPFx web part that loads one plugin
- [ ] Test plugin loading and rendering
- [ ] Verify SharePoint context is accessible

Phase 2: Multi-Plugin Loading

- [ ] Load plugins from `plugins.json` manifest
- [ ] Add plugin navigation UI
- [ ] Implement plugin lifecycle (init, render, dispose)

Phase 3: Shared Services

- [ ] Add event bus
- [ ] Add shared data service
- [ ] Create common API clients

Phase 4: UI Framework

- [ ] Add Fluent UI Web Components
- [ ] Create shared component library
- [ ] Add theming

Phase 5: Real Plugins

- [ ] Site information plugin
- [ ] Site owners plugin
- [ ] Lock/unlock plugin
- [ ] Audit history plugin

Phase 6: Security & Deployment

- [ ] Permission checking
- [ ] Token management
- [ ] Package for app catalog

↑ Back to Top

SPAs vs. True Distributed Applications

↑ Back to Top

7.1 Single Page App (SPA)

↑ Back to Top

Definition: One HTML file, client-side routing, no server-side page loads

Architecture:

```
Browser loads index.html
    ↓
JavaScript takes over navigation
    ↓
URL changes (no reload)
    ↓
Different React components render
```

```
    ↓
Data fetched via APIs as needed
```

Example:

```
SharePoint Site → SPFx Web Part (SPA)
  /home       ← Same HTML, different component
  /sites
  /audit
  /lock
```

Advantages:

- Fast navigation (no reload)
- Rich, interactive UI
- Can work offline (with PWA)
- Easier state management

Disadvantages:

- Initial load can be slow (bundle size)
- Browser back/forward behavior needs management
- SEO challenges (content is JS-rendered)
- Larger client-side bundle

↑ Back to Top

7.2 Progressive Web App (PWA)

↑ Back to Top

Definition: SPA + service worker + offline + installable

Additional Capabilities:

```
SPA
  +
Service Worker (background JS)
  +
manifest.webmanifest (install metadata)
  +
Cache API (offline content)
  +
IndexedDB (offline data)
  =
PWA (installable, offline-capable, app-like)
```

When to Use PWA:

- ✅ Field work (workers need offline)
- ✅ Mobile-first applications
- ✅ High-latency environments
- ❌ Streaming/real-time financial data
- ❌ Applications requiring constant connectivity

For Administrative Tools: PWA is **optional**. Admin tasks typically require connectivity anyway.

↑ Back to Top

7.3 Micro Frontend Architecture

↑ Back to Top

Definition: Federated web applications—multiple independent SPAs composed at runtime

Frameworks:

Framework	Approach	Complexity	SPFx Fit
single-spa	Module federation at runtime	Medium	Good
Module Federation	Webpack 5+ native	Medium	Good

Framework	Approach	Complexity	SPFx Fit
Web Components + host	Browser-native composition	Low	Excellent
iframes	Complete isolation	Low	Good (but complex)

Comparison: Federation vs. Plugins

Aspect	Federation	Plugins
Independence	High - each has own build	Medium - share framework
Deployment	Separate for each module	All together
Complexity	High	Low to Medium
Shared code	Hard	Easy
Communication	Events + pub/sub	Direct + events
Learning curve	Steep	Gentle

Recommendation: For your constraints, **plugins are better than federation**. Avoid Module Federation's complexity unless you have:

- Separate teams per application
- Different tech stacks
- Independent release cycles

↑ Back to Top

Outlook Office Add-Ins: Architecture & Deployment

↑ Back to Top

8.1 Outlook Add-In Model

↑ Back to Top

Outlook Add-In = Web App in a Task Pane

```
Outlook
  │
  ├─ Mail Item
  │    └─ [Your Add-In Icon]
  │
  └─ Task Pane
       └─ Your HTML/CSS/JS
```

Three Types of Outlook Add-Ins:

1. **Task Pane** — Sidebar within Outlook
2. **Message Extension** — Custom commands in ribbon
3. **Module Extension** — Replace Outlook views

Manifest (XML + Modern JSON):

```xml
<!-- outlook-add-in.xml -->
<OfficeApp>
  <Id>12345678-1234-1234-1234-123456789012</Id>
  <Version>1.0.0.0</Version>
  <ProviderName>Site Admin Tools</ProviderName>
  <DefaultLocale>en-US</DefaultLocale>

  <Hosts>
    <Host Name="Mailbox" />
  </Hosts>

  <DefaultSettings>
    <SourceLocation Format="Url">https://yourapp.com/taskpane.html</SourceLocation>
  </DefaultSettings>

  <Permissions>ReadWriteMailbox</Permissions>
</OfficeApp>
```

↑ Back to Top

8.2 Task Pane Add-Ins

↑ Back to Top

Outlook Task Pane (Shared React Code):

```
// taskpane.html
<html>
<head>
  <meta charset="UTF-8" />
  <title>Site Admin</title>
  <script src="https://appsforoffice.microsoft.com/lib/1/hosted/office.js"></script>
  <script src="./shared-app.js"></script>
</head>
<body>
  <div id="root"></div>
  <script src="./outlook-taskpane.js"></script>
</body>
</html>
```

```
// outlook-taskpane.ts
import { createApp } from './shared-app'; // Shared with SPFx

// Outlook-specific context adapter
const outlookContext = {
  user: {
    displayName: Office.context.mailbox.userProfile.displayName,
    email: Office.context.mailbox.userProfile.emailAddress
  },

  // Outlook-specific APIs
  getMailItem: () => Office.context.mailbox.item,

  // Mock SharePoint for Outlook (could call APIs)
  spClient: {
    async getSite(url: string) {
      return fetch(`/api/sites/${url}`).then(r => r.json());
    }
  }
};

const app = createApp(outlookContext);
app.render(document.getElementById('root'));
```

Access to Mail Item:

```
const mailItem = Office.context.mailbox.item;

// Read email properties
console.log(mailItem.subject);
console.log(mailItem.senderEmailAddress);
console.log(mailItem.getEntities()); // Extract metadata

// Modify email
mailItem.subject.setAsync('Updated: ' + mailItem.subject);
```

↑ Back to Top

8.3 Deployment Scenarios

↑ Back to Top

Deployment Methods:

Method	Distribution	Admin Control	Update Frequency	Best For
Centralized Exchange	Admin deploys via Exchange	Full	Manual	Enterprise (GCC)
AppSource	Microsoft store	Microsoft approval	Self-service	Public distribution

Method	Distribution	Admin Control	Update Frequency	Best For
SharePoint Catalog	SharePoint app catalog	IT	Manual	Enterprise
Office 365 ProPlus	With Office installation	Admin	Automatic	Widespread apps
Sideload (Dev)	Manual per machine	User	Manual	Testing

For GCC Compliance:

```
// Deployment via centralized Exchange
PowerShell:

$Credential = Get-Credential
$Session = New-PSSession -ConfigurationName Microsoft.Exchange `
  -ConnectionUri "https://ps.compliance.protection.outlook.com/powershell-liveid/" `
  -Authentication Basic -Credential $Credential

Import-PSSession $Session

New-App -ConfigurationFilePath "C:\outlook-manifest.xml"
```

↑ Back to Top

Markdown & Mermaid Renderer for Legacy Outlook

↑ Back to Top

9.1 Architecture

↑ Back to Top

Use Case: Render Markdown with Mermaid diagrams inside Outlook email/task pane

```
Scenario: Admin sends Markdown email
  "The site lock request has the following status:

   ```mermaid
   graph TD
      A[Request] -->|Pending| B{Approval}
      B -->|Approved| C[Lock]
      B -->|Rejected| D[Deny]
   ```"

Result: Outlook shows rendered diagram, not raw markdown
```

Architecture:

```
┌─────────────────────────────────────┐
│ Email with Markdown + Mermaid       │
├─────────────────────────────────────┤
│ [Open in Outlook Add-In]            │
└─────────────────┬───────────────────┘
                  │
                  ▼
┌─────────────────────────────────────┐
│ Outlook Task Pane Add-In            │
├─────────────────────────────────────┤
│ 1. Extract markdown from item       │
│ 2. Parse markdown                   │
│ 3. Detect code blocks (mermaid)     │
│ 4. Render mermaid diagrams          │
│ 5. Render formatted markdown        │
└─────────────────────────────────────┘
```

↑ Back to Top

9.2 Technical Implementation

↑ Back to Top

Manifest for Markdown Renderer:

```
<OfficeApp>
  <Id>markdown-mermaid-renderer</Id>
  <ProviderName>Markdown Renderer</ProviderName>

  <Hosts>
    <Host Name="Mailbox" />
  </Hosts>

  <DefaultSettings>
    <SourceLocation Format="Url">https://yourapp.com/renderer.html</SourceLocation>
  </DefaultSettings>

  <Permissions>ReadItem</Permissions>
</OfficeApp>
```

Renderer Implementation:

```
// renderer.html
<html>
<head>
  <script src="https://appsforoffice.microsoft.com/lib/1/hosted/office.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/markdown-it@13/dist/markdown-it.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.min.js"></script>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/github-markdown-css@5/github-markdown.css">
</head>
<body>
  <div id="content" class="markdown-body"></div>
  <script src="./renderer.js"></script>
</body>
</html>
```

```
// renderer.ts
Office.onReady(() => {
  const mailItem = Office.context.mailbox.item;

  // Get email body
  mailItem.body.getAsync('text', async (result) => {
    if (result.status === Office.AsyncResultStatus.Succeeded) {
      const markdown = result.value;

      // Parse markdown
      const md = window.markdownit({
        highlight: (code, lang) => {
          // Detect mermaid code blocks
          if (lang === 'mermaid') {
            return `<div class="mermaid">${escapeHtml(code)}</div>`;
          }
          return code;
        }
      });

      const html = md.render(markdown);
      document.getElementById('content').innerHTML = html;

      // Render mermaid diagrams
      mermaid.contentLoaded();
    }
  });
});

function escapeHtml(text) {
  const div = document.createElement('div');
  div.textContent = text;
  return div.innerHTML;
}
```

Styling for Dark/Light Modes:

```
/* Support Outlook's theme */
:root {
  color-scheme: light dark;
}

@media (prefers-color-scheme: dark) {
  body {
    background-color: #1e1e1e;
    color: #e0e0e0;
  }

  .markdown-body {
    background-color: #1e1e1e;
    color: #e0e0e0;
  }

  .mermaid {
    background-color: #2d2d2d;
    border: 1px solid #444;
  }
}

@media (prefers-color-scheme: light) {
  body {
    background-color: #ffffff;
    color: #333333;
  }

  .markdown-body {
    background-color: #ffffff;
  }
}
```

↑ Back to Top

9.3 Key Considerations

↑ Back to Top

Outlook Desktop Client Limitations:

- Task panes have sandboxed environment
- Limited file system access
- No direct HTML editing in compose
- Some APIs unavailable in desktop vs. web

Workaround: Mail Signature Generator

Create a tool that generates HTML-rendered diagrams that users can embed:

```
// Generate diagram as image/HTML for insertion
export async function generateDiagramImage(mermaidCode: string): Promise<Blob> {
  const canvas = await mermaid.mermaidAPI.render(
    'diagram-' + Date.now(),
    mermaidCode
  );

  // Convert to image for embedding
  return new Promise(resolve => {
    canvas.toBlob(blob => resolve(blob));
  });
}
```

Deployment as Mail Extensions:

- **Web Outlook:** Task Pane works directly
- **Outlook Desktop:** Requires add-in manifest deployment
- **Outlook Mobile:** Limited add-in support

↑ Back to Top

Web Components in Modern Browsers

↑ Back to Top

10.1 Web Components Fundamentals

↑ Back to Top

Web Components = 4 Browser Standards:

```
┌─────────────────────────────────────┐
│ Custom Elements                     │
│ define your own HTML tags           │
│ <site-picker></site-picker>         │
└─────────────────────────────────────┘

┌─────────────────────────────────────┐
│ Shadow DOM                          │
│ encapsulate styles and markup       │
│ styles don't leak out               │
└─────────────────────────────────────┘

┌─────────────────────────────────────┐
│ HTML Templates                      │
│ reusable markup templates           │
│ <template> tags                     │
└─────────────────────────────────────┘

┌─────────────────────────────────────┐
│ HTML Imports / ES Modules           │
│ import './my-component.js'          │
└─────────────────────────────────────┘
```

Simple Web Component Example:

```
// my-site-card.js
class SiteCard extends HTMLElement {
  // Declare observed attributes
  static observedAttributes = ['site-id', 'site-title'];

  constructor() {
    super();

    // Attach shadow DOM
    this.attachShadow({ mode: 'open' });
  }

  connectedCallback() {
    // Called when element inserted into DOM
    this.render();
  }

  attributeChangedCallback(name, oldValue, newValue) {
    // Called when observed attribute changes
    this.render();
  }

  render() {
    this.shadowRoot.innerHTML = `
      <style>
        :host {
          display: block;
          padding: 16px;
          border: 1px solid #ccc;
          border-radius: 4px;
        }
        h3 { margin: 0 0 8px; }
      </style>

      <h3>${this.getAttribute('site-title') || 'Untitled'}</h3>
      <p>ID: ${this.getAttribute('site-id')}</p>
    `;
  }

  disconnectedCallback() {
    // Called when element removed from DOM
    // Clean up listeners, timers, etc.
```

```
  }
}

// Register the element
customElements.define('site-card', SiteCard);
```

Usage:

```
<site-card site-id="123" site-title="Sales Hub"></site-card>
```

↑ Back to Top

10.2 Web Components vs. React Components

↑ Back to Top

Comparison:

Aspect	Web Components	React Components
Framework	Browser-native	React library
Reusability	Framework-agnostic	React-only
Learning curve	Moderate	Moderate
Bundle size	~0 KB (native)	~40 KB (React)
Build required	No	Usually
State management	Attributes + properties	State/props
Styling	Shadow DOM (encapsulated)	CSS-in-JS (scoped)
Browser support	Modern browsers	Modern browsers
Development speed	Slower	Faster (for React devs)
Team knowledge	Less common	More common

When to Choose Each:

Choose Web Components if:

- ✅ Plugins must be buildless
- ✅ Different frameworks will use same components
- ✅ Library for third-party consumption
- ✅ Lightweight is priority

Choose React if:

- ✅ Team knows React
- ✅ SPFx integration is primary
- ✅ Rich component library needed
- ✅ Build pipeline available

↑ Back to Top

10.3 SPFx Web Components Support

↑ Back to Top

SPFx Can Host Web Components:

```
// SPFx web part hosting a Web Component
import { BaseClientSideWebPart } from '@microsoft/sp-webpart-base';

export default class MyWebPart extends BaseClientSideWebPart<IMyWebPartProps> {

  protected async onInit(): Promise<void> {
    // Load Web Component
    await import('./my-component.js');
  }

  public render(): void {
    this.domElement.innerHTML = `
```

```
      <my-site-card site-id="123"></my-site-card>
    `;
  }
}
```

Benefits:

- No React overhead if not needed
- Lighter bundle
- Can mix React components + Web Components in same SPFx

↑ Back to Top

10.4 Lit Framework for Web Components

↑ Back to Top

Lit = Lightweight Web Components Framework

```
import { LitElement, html, css } from 'lit';

export class SitePicker extends LitElement {
  static styles = css`
    :host {
      display: block;
      padding: 16px;
    }
    h2 { margin-top: 0; }
  `;

  static properties = {
    selectedSite: { type: Object }
  };

  constructor() {
    super();
    this.selectedSite = null;
  }

  render() {
    return html`
      <h2>Select a Site</h2>
      <input
        placeholder="Enter site name"
        @input="${this._onInput}"
      />
      <button @click="${this._search}">Search</button>
      ${this.selectedSite ? html`
        <div class="selected">
          <p>${this.selectedSite.title}</p>
        </div>
      ` : ''}
    `;
  }

  _onInput(e) {
    this.query = e.target.value;
  }

  _search() {
    // Dispatch custom event
    this.dispatchEvent(new CustomEvent('site-selected', {
      detail: { siteId: '123' }
    }));
  }
}

customElements.define('site-picker', SitePicker);
```

Advantages of Lit:

- Minimal (~5 KB)
- Reactive properties
- Template literals (easier than raw DOM)
- Built-in reactivity

- No virtual DOM overhead
- Can consume without Lit (standard Web Components)

↑ Back to Top

SPFx Platform Support Matrix

↑ Back to Top

11.1 Comprehensive Support Table

↑ Back to Top

Platform	SPFx Version	Release Date	Web Parts	Extensions	Teams	Viva	Stream	Loop	Outlook	OneDrive	Deployment	Admin Control
SharePoint Online	1.23.2 (current)	Sep 2024	✅ Full	✅ Full	Via manifest	N/A	No	No	No	Indirect	App Catalog	Full
SharePoint 2019	1.4.1	Aug 2019	✅ Limited	⚠️ Limited	No	No	No	No	No	N/A	Manual	Limited
Microsoft Teams	1.23.2	Sep 2024	✅ Tabs	❌ No	✅ Native	N/A	No	No	Add-in separate	No	Team App Store	Admin + User
Viva Connections	1.15+	Jan 2021	✅ Cards + Dashboard	✅ Native	N/A	✅ Full	No	No	No	No	App Catalog	Full
Viva Learning	1.18+	May 2021	✅ Content	✅ Extensions	N/A	✅ Full	No	No	No	No	Learning path	Admin
Microsoft Stream	1.20+	Apr 2023	⚠️ Web Experiences	No	N/A	No	✅ Full	No	No	No	Stream portal	Admin
Microsoft Loop	1.22+ (planned)	Future	⚠️ Via API	No	Via manifest	N/A	No	✅ Preview	No	No	Loop workspace	Admin
OneDrive	Not SPFx	N/A	❌ No	✅ File handlers	Indirect	No	No	No	No	✅ File actions	OneDrive UI	Admin
Outlook (Web)	Not SPFx	N/A	❌ No	❌ No	N/A	No	No	No	✅ Add-in	No	ECP / M365 Admin	Admin
Outlook (Desktop)	Not SPFx	N/A	❌ No	❌ No	N/A	No	No	No	✅ Add-in	No	Registry / Exchange	Admin

11.2 Platform-Specific Notes

↑ Back to Top

SharePoint Online

↑ Platform Support Matrix

- **Current SPFx:** 1.23.2 (Node 22, TypeScript 5.x)
- **Web Parts:** ✅ Full support (modern pages only)
- **Extensions:** ✅ Full support (list, item, search results, header/footer)
- **Deployment:** App Catalog (tenant-wide)
- **Admin Settings:**

```
# Enable SPFx web parts
Set-PnPTenant -DisableSpfxOnClassicPages $false

# Configure app permissions
Get-PnPApp | Select Title, Id | Where {$_.Title -like "*"}
```

- **Pros:**
  - Native Microsoft 365 integration
  - Full API access (Graph, SharePoint, Dataverse)
  - Excellent documentation
  - Active community (PnP)
- **Caveats:**
  - Requires Node/npm/TypeScript toolchain

- Build time ~1-2 minutes
- Bundle size impacts performance
- Modern pages only (no classic)

Microsoft Teams

↑ Platform Support Matrix

- **SPFx 1.15+:** Tabs can surface SPFx web parts
- **Deployment:** Teams App Store or sideload
- **Manifest:** `supportedHosts: ["teams"]` in SPFx config
- **Admin Settings:**

```
Teams Admin Center → Manage Apps → Upload to org app catalog
Or use App Installation policies
```

- **Pros:**
  - Single codebase for SharePoint + Teams
  - Reusable components
  - Deep Teams integration
  - Personal tabs and org apps
- **Caveats:**
  - Limited context in Teams (no site context)
  - Requires separate manifest for Teams tab
  - Teams-specific APIs vary by client version
  - Desktop client may lag web features

Viva Connections

↑ Platform Support Matrix

- **SPFx 1.15+:** Dashboard cards and navigation extensions
- **Card Types:**
  - Text card (static)
  - Button card
  - Image card
  - Adaptive card
- **Admin Settings:**

```
Viva Connections → Dashboard → Add new card → Custom
Select SPFx web part
```

- **Pros:**
  - Company-wide dashboard
  - Mobile-first design
  - Deep employee engagement
  - Single entry point
- **Caveats:**
  - Cards have size limits
  - No complex layouts
  - Mobile rendering is critical
  - May require adaptive card JSON

Viva Learning

↑ Platform Support Matrix

- **SPFx 1.18+:** Embed learning content
- **Extensions:** Learning path extensions
- **Deployment:** Via learning provider
- **Pros:**
  - Reaches all employees
  - Integrated with learning platform
  - Skills tracking
- **Caveats:**

    - Content model is limited
    - Must follow Viva format
    - Admin approval required

Microsoft Stream (New)

↑ Platform Support Matrix

- **SPFx 1.20+:** Via web experiences (not web parts directly)
- **Integration:** Stream uses SharePoint backend (Office 365 Groups)
- **Deployment:** Stream portal or SharePoint
- **Pros:**
    - Video platform with Viva integration
    - Shared with entire org
    - Transcript and search included
- **Caveats:**
    - Not traditional SPFx hosting
    - Web experiences API is limited
    - Video player unavailable to customization

Microsoft Loop

↑ Platform Support Matrix

- **Status:** SPFx support planned (not yet GA)
- **Planned Integration:** Loop components can reference SPFx
- **Deployment:** Loop workspace
- **Future Pros:**
    - Real-time collaboration
    - Embedded components
    - Live data binding
- **Current Caveats:**
    - Still in preview
    - APIs not finalized
    - Limited SPFx integration documentation

OneDrive for Business

↑ Platform Support Matrix

- **Note:** Not an SPFx host, but integration is possible
- **Method:** File handlers + extensions
- **Deployment:** Manifest via admin portal
- **Pros:**
    - File-level operations
    - Rich previews
    - Widespread access
- **Caveats:**
    - OneDrive is SharePoint-backed
    - Limited UI customization
    - File context only

Outlook (Web & Desktop)

↑ Platform Support Matrix

- **Note:** Use Office Add-ins, not SPFx
- **Types:** Task Pane, Message Extensions, Module Extensions
- **Deployment Methods:**
    - Exchange Admin Center
    - AppSource
    - Centralized deployment
- **Admin Settings (GCC):**

```
Exchange Online PowerShell:
New-App -ConfigurationFilePath "C:\manifest.xml"
```

- **Pros:**
  - Reaches all email users
  - Works in desktop client
  - Task pane is rich interface
  - Shared code with SPFx possible
- **Caveats:**
  - Separate manifest/toolchain from SPFx
  - Desktop client lags web
  - Limited mailbox API access
  - Mail read-only by default

Architectural Decision Matrix

↑ Back to Top

Use This Matrix to Choose Your Deployment Platform:

Requirement	SharePoint	Teams	Viva	Outlook	Loop
Site administration	✅ Best	⚠️ Via teams	❌ No	❌ No	❌ No
Company-wide visibility	✅ Via hub	✅ Via pins	✅ Best	❌ No	⚠️ Future
Mobile-first	⚠️ Limited	✅ Good	✅ Best	⚠️ Outlook app	✅ Future
Real-time collaboration	❌ No	✅ Good	⚠️ Basic	❌ No	✅ Best
Email context	❌ No	❌ No	❌ No	✅ Best	❌ No
Data entry form	✅ Good	✅ Good	⚠️ Cards only	⚠️ Task pane	⚠️ Future
Dashboard/Analytics	✅ Good	✅ Good	✅ Best	❌ No	❌ No
Workflow trigger	✅ Good	✅ Good	⚠️ Limited	⚠️ Task pane	⚠️ Future
Build complexity	High	High	High	Medium	Unknown
Deployment friction	Low	Low	Low	Medium	Unknown

↑ Back to Top

Recommended Implementation Path

↑ Back to Top

Phase 1: Proof of Concept (Weeks 1-2)

- [ ] Create minimal SPFx web part (no plugins yet)
- [ ] Load one static plugin from JSON
- [ ] Verify SharePoint context access
- [ ] Test in SharePoint and Teams

Phase 2: Plugin Infrastructure (Weeks 3-4)

- [ ] Build event bus
- [ ] Implement plugin loader
- [ ] Create shared context adapter
- [ ] Add error handling and logging

Phase 3: Shared Services (Weeks 5-6)

- [ ] Build SharePoint API client
- [ ] Build Graph API client
- [ ] Implement data service
- [ ] Add authentication layer

Phase 4: Real Plugins (Weeks 7-9)

- [ ] Site information plugin
- [ ] Site permissions plugin
- [ ] Audit history plugin
- [ ] Add Web Components UI

Phase 5: Expand Platforms (Weeks 10-12)

- [ ] Teams tab integration
- [ ] Outlook task pane (shared code)
- [ ] Web deployment
- [ ] Documentation

Phase 6: Production Hardening (Weeks 13-16)

- [ ] Security audit
- [ ] Performance optimization
- [ ] Load testing
- [ ] Deployment automation
- [ ] App catalog packaging

↑ Back to Top

Technical Glossary (Addendum)

↑ Back to Top

Term	Definition	Relevance
Adaptive Card	Microsoft's cross-platform card format	Viva, Teams, Outlook
App Catalog	SharePoint repo for SPFx solutions	Deployment
Attribute	Custom HTML property (e.g., `<my-el attr="value">`)	Web Components
BroadcastChannel	Browser API for cross-tab messaging	Shared data sync
Cache API	Service worker caching mechanism	PWA, offline
Custom Elements	Browser API for registering custom HTML tags	Web Components
Dataverse	Microsoft's cloud data platform	Power Platform integration
Esbuild	Fast JavaScript bundler	Plugin compilation
Extension	SPFx-hosted code extending SharePoint UI	Search, list, header
Graph API	Microsoft's unified Microsoft 365 API	Auth, user, teams, mail
Heft	Monorepo build tool from Microsoft	SPFx 1.22+ toolchain
IndexedDB	Browser database for structured data	Large offline cache
Jotai	Atomic state management for React	State if using React
Lit	Minimal Web Components framework	Lightweight UI
MCP	Model Context Protocol	Interop between tools
Module Federation	Webpack 5 feature for federated apps	Microfrontends
MSAL	Microsoft Authentication Library	Browser auth
PCF	Power Apps Component Framework	Power Apps custom controls
PnP JS	Community JavaScript library wrappers	SharePoint/Graph access
PWA	Progressive Web App (installable, offline)	Offline-first apps
Rig	Reusable build configuration (Heft)	Toolchain standardization
RxJS	Reactive programming library	Observable streams
Shadow DOM	Encapsulated DOM subtree	Web Components isolation
Single-spa	JavaScript framework for micro frontends	Federated architecture
SPA	Single Page Application (client-side routing)	All web apps here
SPFx	SharePoint Framework	Deployment platform
Task Pane	Sidebar UI in Office Add-ins	Outlook, Word, Excel
Viva	Microsoft's employee experience platform	Internal comms/learning
Web Components	Browser-native component model	Framework-agnostic UI
Yeoman	Scaffolding generator (legacy SPFx)	Project initialization
Zustand	Minimal state management for React	Lighter than Redux

↑ Back to Top

Summary & Next Steps

↑ Back to Top

This Addendum Covered:

1. ✅ **Pluggable Architecture** — Event bus, shared data, inter-plugin communication
2. ✅ **Security & Context** — Isolation, permissions, token management
3. ✅ **TypeScript Compilation** — Centralized vs. distributed build strategies
4. ✅ **Fluent UI Analysis** — React vs. Web Components trade-offs
5. ✅ **Minimal Starting Point** — Single applet proof of concept
6. ✅ **SPA/PWA/Micro Frontend** — Architectural patterns explained
7. ✅ **Outlook Add-Ins** — Task pane + Markdown/Mermaid renderer
8. ✅ **Web Components** — Standards, Lit framework, SPFx integration
9. ✅ **Platform Matrix** — Complete SPFx support across 9 Microsoft products

Recommended Next Steps:

1. **Validate Approach** — Review with IT/security teams
2. **Create Simple POC** — Single plugin, basic event bus
3. **Test in Teams** — Ensure Teams tab manifest works
4. **Expand Services** — Add real SharePoint/Graph clients
5. **Plan Outlook** — Decide on task pane vs. message extensions
6. **Design UI** — Choose Fluent React or Web Components
7. **Scale Plugins** — Start with site admin, expand from there

---

End of Addendum

For questions or updates, refer to the main SPFx.Launcher.Plan.Preview.md document.